

Why an **agentic** flow, not a keyword-matched script

Two decisions came before any flow was drawn: whether to use AI at all, then what shape it should take. A keyword bot only fires on triggers it expects; *"nah it's definitely not out there, asked next door too"* means "already checked," but no trigger list catches it. A free-form AI chatbot swings too far the other way — it can be talked into promising refunds or inventing policy. The agentic loop is the middle path: Gemini reads intent like a free-form model would, but every fact and every move it's *allowed* to make still comes from the app.

KEYWORD-ONLY

- ✗ Rigid triggers, misses anything unscripted
- ✗ No read on intent or tone
- ✗ Kept here as the offline fallback

FREE-FORM AI CHATBOT

- ✗ Understands phrasing — but can promise anything
- ✗ Facts and policy come from the model itself
- ✗ One good jailbreak away from a bad answer

AGENTIC LOOP (THIS APP)

- ✓ Understands phrasing, same as free-form
- ✓ One call picks one move from a legal list
- ✓ Facts always come from the app, not the model

Intent layer

Function-calling, `mode: "ANY"` — one move, chosen only from what's legal at the customer's current step.

Guardrails

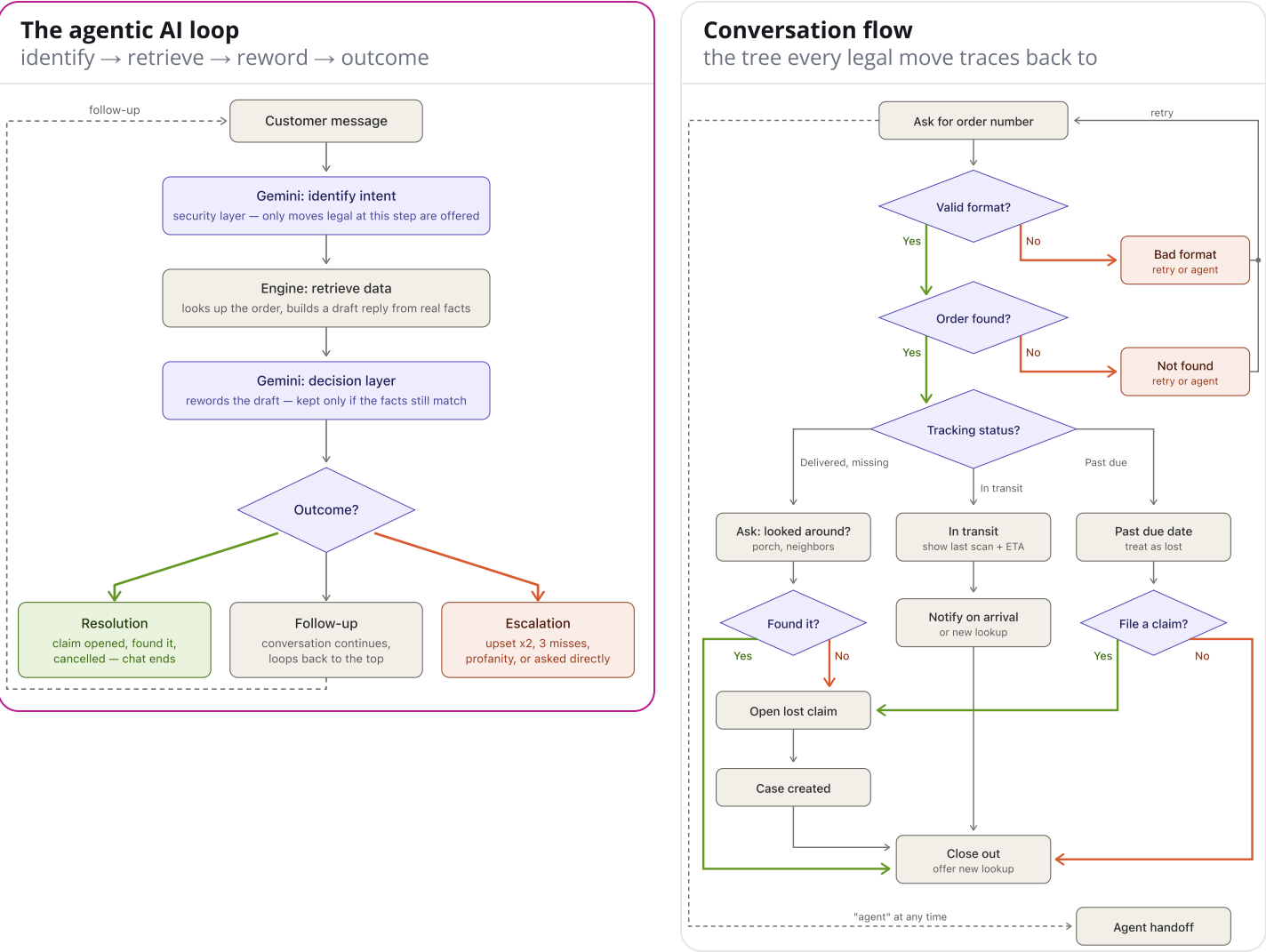
The server re-validates the move regardless of what the model returned. An illegal pick is rejected, not trusted.

Fact isolation

Gemini never holds facts — only rewords a draft the engine built from real data, checked afterward for drift.

Input fencing

Customer text is fenced as data in the prompt, not instructions — "ignore your rules..." is just a string to read.



What it took to build it, given those design choices

HTML/CSS/JS frontend → Node/Express API (proxy server) → fine-tuning the AI's behavior — that third phase is most of what's below

Languages & runtime

- **HTML/CSS/vanilla JS** build the one-page frontend — no framework, no build step, just `<script>` tags
- **Node.js 18+** runs `server.js` as the Express proxy
- `engine.js` — the exact same file — is `require()`'d server-side and `<script>`-loaded client-side, so both "brains" run identical logic

Services & hosting

- **Google Gemini API** — called twice a turn via server-side `fetch`, key attached as an `x-goog-api-key` header, never from the browser
- **Render** — `server.js` runs there as its own web service, auto-deployed straight from GitHub on every push
- **GitHub** — hosts the public repo Render and any reviewer both pull from

Frameworks & libraries

- **Express** defines every REST route and runs CORS + rate-limit middleware before a request ever reaches the flow engine
- **Playwright** drives real Chrome to click through the UI, screenshot each state, and catch layout regressions; it also renders this deck to PDF
- Node's built-in test runner — `node --test` runs all 129 tests, no extra dependency

CS methodologies & patterns

- **Finite state machine** — one `STATES` object keyed by name; the whole conversation is just `s.state` plus a few fields
- **Constrained decoding** — Gemini's function-calling enum is rebuilt from `allowedActions(s)` every turn, so illegal moves are never offered
- **Proxy + graceful degradation** — `server.js` alone holds the key; a try/catch around the Gemini call falls through to the same rule engine the browser itself falls back to

Data layer

- `orders.json` / `users.json` — hand-written, loaded once with `require()`, queried as plain in-memory arrays
- One-to-many enforced by convention (each order carries a `userId`) and checked by a dedicated test, not a schema

Testing & verification

- **129 tests** run directly against the Express app and the engine module — real HTTP calls, no mocks
- Playwright screenshots the running app and measures bounding boxes to catch visual regressions
- Dedicated tests send malformed origins and message floods to prove CORS and rate limits actually reject them

Edge cases, security, and being honest about scale

Edge cases INPUT

31 mock orders, 11 statuses surfaced gaps a happy-path demo wouldn't:

- A new order number mid-chat was silently dropped — now it asks before switching
- Misdelaivered orders were told to "check the door" — now they open a claim directly
- "already checked next door" read as unclear to keywords — the agent reads it as an answer

Security MODEL

A model that can talk freely can be talked into promising things:

- Customer text is fenced as data, not instructions — "ignore your rules..." is inert
- Function calling restricts Gemini to a legal move list; the server re-checks anyway
- Order numbers from the model are validated and looked up, never trusted alone
- A reworded reply that changes a number, date, or id is discarded for the plain draft

Proxy server KEY

This repo is public, so the key can never ship in anything downloaded:

- `server.js` alone holds or sends the key — browser calls it, never Gemini
- **Render hosts that proxy**, key as a private env var, never committed to GitHub
- CORS allowlists origins; rate limits stop a script that ignores CORS anyway
- A compromised frontend has nothing to leak — the key was never there

Stale paths & loops FLOW

A confused customer or a wrong pick shouldn't be able to loop forever:

- 12 messages in one chat → forced offer to escalate or restart
- 3 unmatched replies, or 2 upset messages, trigger the same offer
- Restart wipes server *and* visible state — no half-remembered session

Simulating scale SCOPE

A take-home can't stand up real infrastructure, but it has to behave like it did:

- `orders.json` / `users.json` stand in for a database and auth system, with the one-to-many shape a real one would have
- No ORM, no query language — just `require()` and array lookups
- Claims and alerts are logged as sent, not actually sent — no real email/SMS service
- The README names exactly what's simulated — the tradeoff is explicit, not hidden

What I'd fix and add — scoped out by time and scale, not overlooked

To add later

- **A real database and real auth with full CRUD**, replacing orders.json / users.json — today's data is read-only lookups; a real backend needs to create, update, and delete orders and accounts too, with hashed, salted passwords checked only server-side.
- **Real carrier lookups and richer API calls** — live USPS/UPS/FedEx tracking and delay data, plus the create/update/cancel calls a real claim needs, not just a read-only lookup.
- **A true handoff** by phone or live chat, with the full transcript passed along so customers never repeat themselves.
- **Claims and alerts that actually happen**: file the claim, send the email or SMS.
- **More connectors and tools** — CRM, payment/refund processors, specialized agents behind each one's own API, wired in as scoped calls rather than flow rewrites.

To work on

- **1-2 backup AI providers** ahead of the keyword fallback — a second model or provider before falling all the way back, keyword matching as the true last resort.
- **Configurability** — tone/persona and guardrail parameters (miss limit, upset threshold, escalation wording) as settings, not hardcoded constants.
- **Check meaning, not just numbers**, when validating Gemini's rewording.
- **Wire up the analytics already designed in**: drop-off by step, escalation rate, resolution-without-a-human rate, straight from the state transitions `engine.js` already logs.